STA 445 S26 Assignment 6

2026-03-24

Load your packages here:

Problem 1

A common task is to take a set of data that has multiple categorical variables and create a table of the number of cases for each combination. An introductory statistics textbook contains a data set summarizing student surveys from several sections of an intro class. The two variables of interest are `Gender` and `Year` which are the students gender and year in college. *Note: you will need to refer to Chapter 4 and Chapter 7 for some of the operations needed below - this is a great time to review chapter 4!*

a) Download the data set using the following:

```
Survey <- read.csv('https://www.lock5stat.com/datasets2e/StudentSurvey.csv',
                   na.strings=c('',' '))
```

b) Select the specific columns of interest **Year** and **Gender**

```
Survey <- Survey |>
  select(Year, Gender)
head(Survey)
```

```
##        Year Gender
## 1    Senior      M
## 2 Sophomore      F
## 3 FirstYear      M
## 4    Junior      M
## 5 Sophomore      F
## 6 Sophomore      F
```

c) Convert the **Year** column to factors and properly order the factors based on common US progression (FirstYear - Sophomore - Junior - Senior)

```
Survey <- Survey |>
  mutate(
    Year = factor(Year,
                  levels = c("FirstYear", "Sophomore", "Junior", "Senior"),
                  ordered = TRUE))
```

d) Convert the **Gender** column to factors and rename them Male/Female.

```
Survey <- Survey |>
  mutate(
    Gender = factor(Gender,
                    levels = c("M", "F"),
```

```r
                     labels = c("Male", "Female")))
str(Survey)
```

```
## 'data.frame':    362 obs. of  2 variables:
##  $ Year  : Ord.factor w/ 4 levels "FirstYear"<"Sophomore"<..: 4 2 1 3 2 2 1 2 3 1 ...
##  $ Gender: Factor w/ 2 levels "Male","Female": 1 2 1 1 2 2 2 1 2 2 ...
```

e) Produce a data set with eight rows and three columns that contains the number of responses for each gender:year combination. *You might want to look at the following functions:* ***`dplyr::count`*** *and* ***`dplyr::drop_na`****.*

```r
Survey_count <- Survey |>
  drop_na() |>
  count(Year, Gender, .drop = FALSE) |>
  arrange(Year, Gender)
Survey_count
```

```
##        Year Gender  n
## 1 FirstYear   Male 51
## 2 FirstYear Female 43
## 3 Sophomore   Male 99
## 4 Sophomore Female 96
## 5    Junior   Male 17
## 6    Junior Female 18
## 7    Senior   Male 26
## 8    Senior Female 10
```

f) Pivot the table in part (e) to produce a table of the number of responses in the following form:

Gender	First Year	Sophomore	Junior	Senior
Female				
Male				

```r
Survey_count |>
  pivot_wider(names_from = Year, values_from =  n)
```

```
## # A tibble: 2 x 5
##   Gender FirstYear Sophomore Junior Senior
##   <fct>      <int>     <int>  <int>  <int>
## 1 Male          51        99     17     26
## 2 Female        43        96     18     10
```

Problem 2

For this problem we will consider two simple data sets. Remember to erase eval = FALSE from the code chunk below.

```r
    A <- tribble(
      ~Name, ~Car,
      'Alice', 'Ford F150',
      'Bob',   'Tesla Model III',
```

```r
      'Charlie', 'VW Bug')

    B <- tribble(
      ~First.Name, ~Pet,
      'Bob',  'Cat',
      'Charlie', 'Dog',
      'Alice', 'Rabbit')
```

a. Squish the data frames together to generate a data set with three rows and three columns. Do two ways: first using `cbind` and then using one of the `dplyr join` commands.

```r
cbind(A, B) #does not work the way we want
```

```
##      Name             Car First.Name    Pet
## 1   Alice       Ford F150        Bob    Cat
## 2     Bob Tesla Model III    Charlie    Dog
## 3 Charlie          VW Bug      Alice Rabbit
```

```r
full_join(A, B, by = c("Name" = "First.Name")) #does work the way we want
```

```
## # A tibble: 3 x 3
##   Name    Car             Pet
##   <chr>   <chr>           <chr>
## 1 Alice   Ford F150       Rabbit
## 2 Bob     Tesla Model III Cat
## 3 Charlie VW Bug          Dog
```

b. It turns out that Alice also has a pet guinea pig. Add another row to the `B` data set. Do this using either the base function `rbind`, or either of the `dplyr` functions `add_row` or `bind_rows`.

```r
BB <- add_row(B, First.Name = "Alice", Pet = "Guinea Pig")
BB
```

```
## # A tibble: 4 x 2
##   First.Name Pet
##   <chr>      <chr>
## 1 Bob        Cat
## 2 Charlie    Dog
## 3 Alice      Rabbit
## 4 Alice      Guinea Pig
```

c. Squish the `A` and `B` data sets together to generate a data set with four rows and three columns. Do this two ways: first using `cbind` and then using one of the `dplyr join` commands. Which was easier to program? Which is more likely to have an error.

```r
cbind(A, BB) #this does not work due to the column names being different
```

```r
full_join(A, BB, by = c("Name" = "First.Name"))
```

```
## # A tibble: 4 x 3
##   Name    Car             Pet
##   <chr>   <chr>           <chr>
## 1 Alice   Ford F150       Rabbit
## 2 Alice   Ford F150       Guinea Pig
```

```
## 3 Bob     Tesla Model III Cat
## 4 Charlie VW Bug          Dog
```

Using the dplyr join commands is easier as they are more accommodating for things such as column name differences, cbind is very basic and does not allow anything abnormal to work the way you would want.

Problem 3

From this book's GitHub there is a .csv file of the daily maximum temperature in Flagstaff at the Pulliam Airport. The link is: https://raw.githubusercontent.com/BuscagliaR/STA_444_v2/master/data-raw/FlagMaxTemp.csv

a) Create a line graph that gives the daily maximum temperature for 2005. *Make sure the x-axis is a date and covers the whole year.*

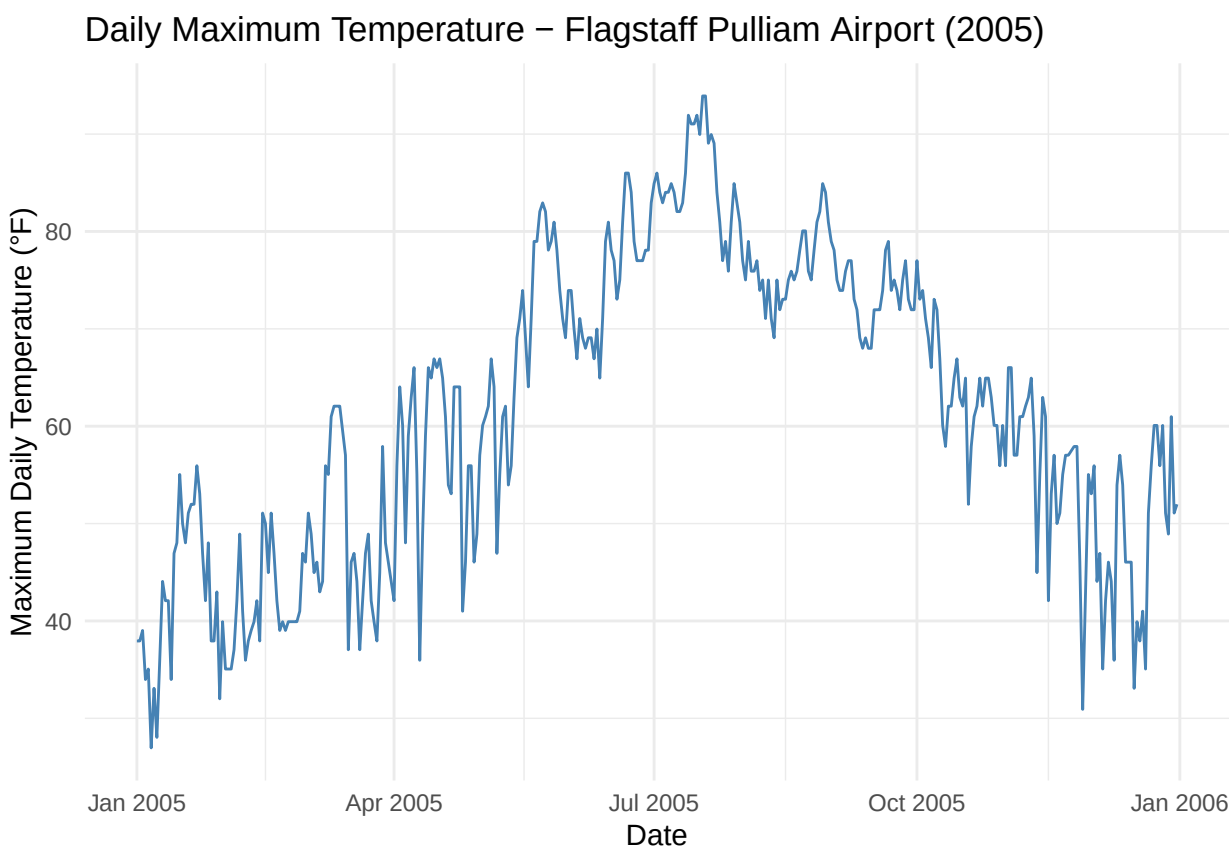
```r
flag_temp <- read_csv(
  "https://raw.githubusercontent.com/BuscagliaR/STA_444_v2/master/data-raw/FlagMaxTemp.csv")
#view(flag_temp)
```

```r
flag_long <- flag_temp |>
  pivot_longer(
    cols = `1`:`31`,
    names_to = "Day",
    values_to = "MaxTemp") |>
  mutate(
    Day = as.integer(Day),
    Date = make_date(Year, Month, Day)) |>
  filter(!is.na(MaxTemp)) |>
  select(Year, Month, Day, MaxTemp, Date)
#view(flag_long)
```

```r
flag_2005 <- flag_long |>
  filter(Year == 2005)
head(flag_2005)
```

```
## # A tibble: 6 x 5
##    Year Month   Day MaxTemp Date
##   <dbl> <dbl> <int>   <dbl> <date>
## 1  2005     1     1    37.9 2005-01-01
## 2  2005     1     2    37.9 2005-01-02
## 3  2005     1     3    39.0 2005-01-03
## 4  2005     1     4    34.0 2005-01-04
## 5  2005     1     5    35.1 2005-01-05
## 6  2005     1     6    27.0 2005-01-06
```

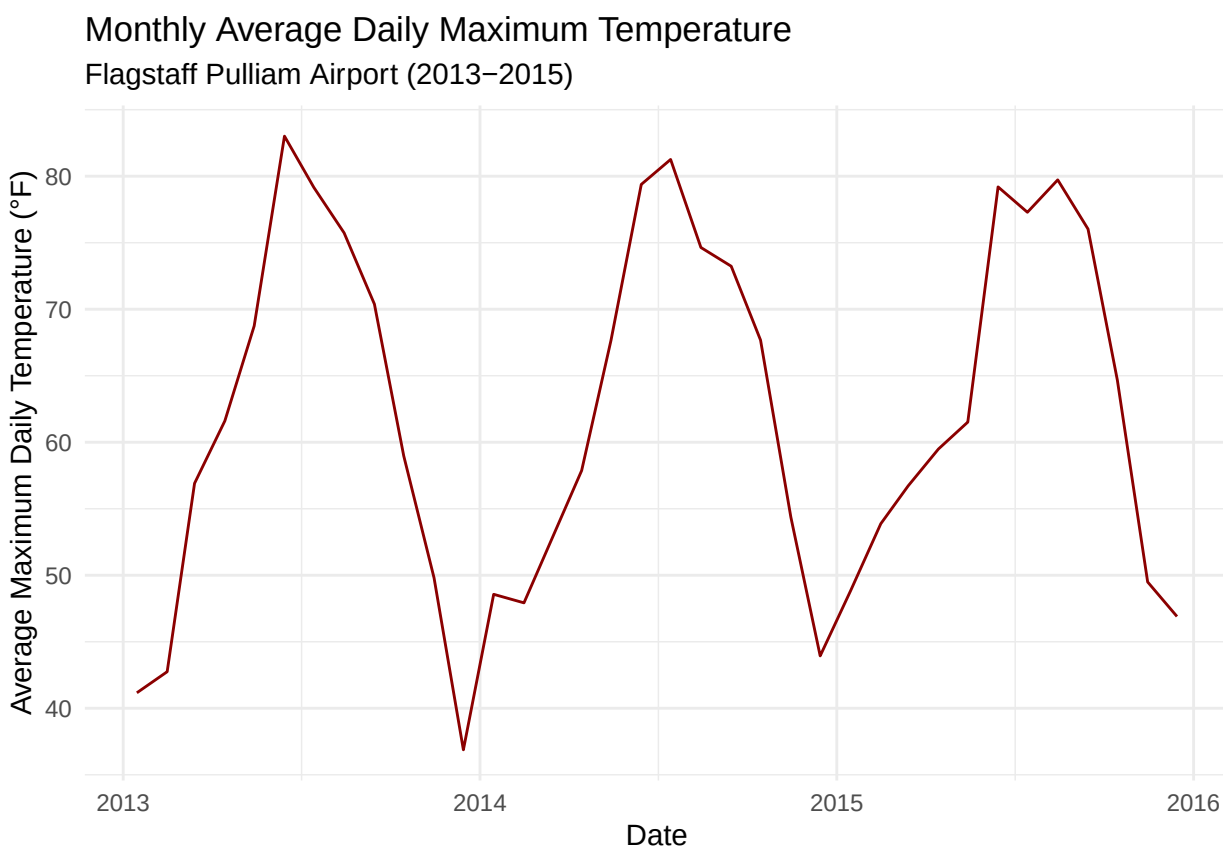
```r
ggplot(flag_2005, aes(x = Date, y = MaxTemp)) +
  geom_line(color = "steelblue") +
  labs(
    title = "Daily Maximum Temperature - Flagstaff Pulliam Airport (2005)",
    x = "Date",
    y = "Maximum Daily Temperature (°F)") +
  theme_minimal()
```



b) Create a line graph that gives the monthly average maximum temperature for 2013 - 2015. *Again the x-axis should be the date and span 3 years.*

```
monthly_avg <- flag_long |>
  filter(Year %in% 2013:2015) |>
  group_by(Year, Month) |>
  summarise(
    AvgMaxTemp = mean(MaxTemp, na.rm = TRUE),
    .groups = "drop") |>
  mutate(
    Date = make_date(Year, Month, 15)) #this is for plotting idk
```

```
ggplot(monthly_avg, aes(x = Date, y = AvgMaxTemp)) +
  geom_line(color = "darkred") +
  labs(
    title = "Monthly Average Daily Maximum Temperature",
    subtitle = "Flagstaff Pulliam Airport (2013-2015)",
    x = "Date",
    y = "Average Maximum Daily Temperature (°F)") +
  theme_minimal()
```



Problem 4

The package `nycflights13` contains information about all the flights that arrived in or left from New York City in 2013. This package contains five data tables, but there are three data tables we will work with. The data table `flights` gives information about a particular flight, `airports` gives information about a particular airport, and `airlines` gives information about each airline. Create a table of all the flights on February 14th by Virgin America that has columns for the carrier, destination, departure time, and flight duration. Join this table with the airports information for the destination. Notice that because the column for the destination airport code doesn't match up between `flights` and `airports`, you'll have to use the `by=c("TableA.Col"="TableB.Col")` argument where you insert the correct names for `TableA.Col` and `TableB.Col`.

```
feb14_va <- flights |>
  filter(month == 2, day == 14, carrier == "VX") |>
  select(carrier, dest, dep_time, air_time) |>
  left_join(airports,
            by = c("dest" = "faa")) |>
  select(carrier,
         destination = name,
         departure_time = dep_time,
         flight_duration = air_time,
         dest_airport = dest)
head(feb14_va)
```

```
## # A tibble: 6 x 5
##   carrier destination        departure_time flight_duration dest_airport
```

```
##   <chr>   <chr>                        <int>           <dbl> <chr>
## 1 VX      Los Angeles Intl               706             347 LAX
## 2 VX      San Francisco Intl             732             344 SFO
## 3 VX      Los Angeles Intl               909             341 LAX
## 4 VX      Mc Carran Intl                 934             307 LAS
## 5 VX      San Francisco Intl            1029             351 SFO
## 6 VX      Los Angeles Intl              1317             349 LAX
```

Problem 5

Data table joins are extremely common because effective database design almost always involves having multiple tables for different types of objects. To illustrate both table joins and the usefulness of multiple tables we will develop a set of data frames that will represent a credit card company's customer data base. We will have tables for Customers, Retailers, Cards, and Transactions. Below is code that will create and populate these tables.

```r
Customers <- tribble(
  ~PersonID, ~Name, ~Street, ~City, ~State,
  1, 'Derek Sonderegger',  '231 River Run', 'Flagstaff', 'AZ',
  2, 'Aubrey Sonderegger', '231 River Run', 'Flagstaff', 'AZ',
  3, 'Robert Buscaglia', '754 Forest Heights', 'Flagstaff', 'AZ',
  4, 'Roy St Laurent', '845 Elk View', 'Flagstaff', 'AZ')

Retailers <- tribble(
  ~RetailID, ~Name, ~Street, ~City, ~State,
  1, 'Kickstand Kafe', '719 N Humphreys St', 'Flagstaff', 'AZ',
  2, 'MartAnnes', '112 E Route 66', 'Flagstaff', 'AZ',
  3, 'REI', '323 S Windsor Ln', 'Flagstaff', 'AZ' )

Cards <- tribble(
  ~CardID, ~PersonID, ~Issue_DateTime, ~Exp_DateTime,
  '9876768717278723',  1,  '2019-9-20 0:00:00', '2022-9-20 0:00:00',
  '5628927579821287',  2,  '2019-9-20 0:00:00', '2022-9-20 0:00:00',
  '7295825498122734',  3,  '2019-9-28 0:00:00', '2022-9-28 0:00:00',
  '8723768965231926',  4,  '2019-9-30 0:00:00', '2022-9-30 0:00:00' )

Transactions <- tribble(
  ~CardID, ~RetailID, ~DateTime, ~Amount,
  '9876768717278723', 1, '2019-10-1 8:31:23',    5.68,
  '7295825498122734', 2, '2019-10-1 12:45:45',  25.67,
  '9876768717278723', 1, '2019-10-2 8:26:31',    5.68,
  '9876768717278723', 1, '2019-10-2 8:30:09',    9.23,
  '5628927579821287', 3, '2019-10-5 18:58:57',  68.54,
  '7295825498122734', 2, '2019-10-5 12:39:26',  31.84,
  '8723768965231926', 2, '2019-10-10 19:02:20', 42.83)

Cards <- Cards %>%
  mutate( Issue_DateTime = lubridate::ymd_hms(Issue_DateTime),
          Exp_DateTime   = lubridate::ymd_hms(Exp_DateTime) )
Transactions <- Transactions %>%
```

```
  mutate( DateTime = lubridate::ymd_hms(DateTime))
```

a) Create a table that gives the credit card statement for Derek. It should give all the transactions, the amounts, and the store name. Write your code as if the only initial information you have is the customer's name. *Hint: Do a bunch of table joins, and then filter for the desired customer name. To be efficient, do the filtering first and then do the table joins.*

```
derek_statement <- Customers |>
  filter(Name == "Derek Sonderegger") |>
  inner_join(Cards, by = "PersonID") |>
  inner_join(Transactions, by = "CardID") |>
  inner_join(Retailers %>% rename(Store = Name), by = "RetailID") |>
  select(
    Transaction_DateTime = DateTime,
    Amount,
    Store) |>
  arrange(Transaction_DateTime)

derek_statement
```

```
## # A tibble: 3 x 3
##   Transaction_DateTime Amount Store
##   <dttm>                <dbl> <chr>
## 1 2019-10-01 08:31:23    5.68 Kickstand Kafe
## 2 2019-10-02 08:26:31    5.68 Kickstand Kafe
## 3 2019-10-02 08:30:09    9.23 Kickstand Kafe
```

b) Aubrey has lost her credit card on Oct 15, 2019. Close her credit card at 4:28:21 PM and issue her a new credit card in the `Cards` table. *Hint: Using the Aubrey's name, get necessary CardID and PersonID and save those as* ***`cardID`*** *and* ***`personID`****. Then update the* ***`Cards`*** *table row that corresponds to the* ***`cardID`*** *so that the expiration date is set to the time that the card is closed. Then insert a new row with the* ***`personID`*** *for Aubrey and a new* ***`CardID`*** *number that you make up.*

```
aubrey_info <- Customers |>
  filter(Name == "Aubrey Sonderegger") |>
  inner_join(Cards, by = "PersonID")

personID <- aubrey_info$PersonID
cardID <- aubrey_info$CardID

close_time <- ymd_hms("2019-10-15 16:28:21")

Cards <- Cards |>
  mutate(
    Exp_DateTime = if_else(CardID == cardID, close_time, Exp_DateTime))

new_cardID <- "4629183750293847"

Cards <- Cards |>
  add_row(
    CardID = new_cardID,
```

```
    PersonID = personID,
    Issue_DateTime = close_time,
    Exp_DateTime = ymd_hms("2022-10-15 00:00:00"))

Cards
```

```
## # A tibble: 5 x 4
##   CardID           PersonID Issue_DateTime      Exp_DateTime
##   <chr>               <dbl> <dttm>              <dttm>
## 1 9876768717278723        1 2019-09-20 00:00:00 2022-09-20 00:00:00
## 2 5628927579821287        2 2019-09-20 00:00:00 2019-10-15 16:28:21
## 3 7295825498122734        3 2019-09-28 00:00:00 2022-09-28 00:00:00
## 4 8723768965231926        4 2019-09-30 00:00:00 2022-09-30 00:00:00
## 5 4629183750293847        2 2019-10-15 16:28:21 2022-10-15 00:00:00
```

c) Aubrey is using her new card at Kickstand Kafe on Oct 16, 2019 at 2:30:21 PM for coffee with a charge of $4.98. Generate a new transaction for this action. *Hint: create temporary variables* ***card****,****retailid****,****datetime****, and* ***amount*** *that contain the information for this transaction and then write your code to use those. This way in the next question you can just use the same code but modify the temporary variables. Alternatively, you could write a function that takes in these four values and manipulates the tables in the GLOBAL environment using the* `<<-` *command to assign a result to a variable defined in the global environment. The reason this is OK is that in a real situation, these data would be stored in a database and we would expect the function to update that database.*

```
card <- new_cardID
retailid <- 1
datetime <- ymd_hms("2019-10-16 14:30:21")
amount <- 4.98

Transactions <- Transactions |>
  add_row(
    CardID = card,
    RetailID = retailid,
    DateTime = datetime,
    Amount = amount)

Transactions
```

```
## # A tibble: 8 x 4
##   CardID           RetailID DateTime            Amount
##   <chr>               <dbl> <dttm>               <dbl>
## 1 9876768717278723        1 2019-10-01 08:31:23   5.68
## 2 7295825498122734        2 2019-10-01 12:45:45  25.7
## 3 9876768717278723        1 2019-10-02 08:26:31   5.68
## 4 9876768717278723        1 2019-10-02 08:30:09   9.23
## 5 5628927579821287        3 2019-10-05 18:58:57  68.5
## 6 7295825498122734        2 2019-10-05 12:39:26  31.8
## 7 8723768965231926        2 2019-10-10 19:02:20  42.8
## 8 4629183750293847        1 2019-10-16 14:30:21   4.98
```

d) On Oct 17, 2019, some nefarious person is trying to use her OLD credit card at REI. Make sure your code in part (c) first checks to see if the credit card is active before creating a new transaction. Using

the same code, verify that the nefarious transaction at REI is denied. *Hint: your check ought to look something like this:*

```
card <- '9876768717278723'
retailid <- 2
datetime <- ymd_hms('2019-10-16 14:30:21')
amount <- 4.98

# If the card is currently valid, this should return exactly 1 row.
Valid_Cards <- Cards %>%
  filter(CardID == card, Issue_DateTime <= datetime, datetime <= Exp_DateTime)

# If the transaction is valid, insert the transaction into the table
if( nrow(Valid_Cards) == 1){
  # Some code to insert the transaction
}else{
  print('Card Denied')
}
```

```
card <- "5628927579821287" #old card
retailid <- 3
datetime <- ymd_hms("2019-10-17 12:00:00") #any time after close?
amount <- 123.45

Valid_Cards <- Cards |>
  filter(
    CardID == card,
    Issue_DateTime <= datetime,
    datetime <= Exp_DateTime)

if (nrow(Valid_Cards) == 1) {
  Transactions <- Transactions |>
    add_row(
      CardID = card,
      RetailID = retailid,
      DateTime = datetime,
      Amount = amount)
  print("Transaction approved and added")
} else {
  print("Card Denied")
}
```

```
## [1] "Card Denied"
```

```
Transactions
```

```
## # A tibble: 8 x 4
##   CardID           RetailID DateTime            Amount
##   <chr>               <dbl> <dttm>               <dbl>
## 1 9876768717278723        1 2019-10-01 08:31:23   5.68
## 2 7295825498122734        2 2019-10-01 12:45:45  25.7
## 3 9876768717278723        1 2019-10-02 08:26:31   5.68
```

```
## 4 9876768717278723        1 2019-10-02 08:30:09   9.23
## 5 5628927579821287        3 2019-10-05 18:58:57  68.5
## 6 7295825498122734        2 2019-10-05 12:39:26  31.8
## 7 8723768965231926        2 2019-10-10 19:02:20  42.8
## 8 4629183750293847        1 2019-10-16 14:30:21   4.98
```

e) Generate a table that gives the credit card statement for Aubrey. It should give all the transactions, amounts, and retailer name for both credit cards she had during this period.

```r
aubrey_statement <- Customers |>
  filter(Name == "Aubrey Sonderegger") |>
  inner_join(Cards, by = "PersonID") |>
  inner_join(Transactions, by = "CardID") |>
  inner_join(Retailers %>% rename(Store = Name),
             by = "RetailID") |>
  select(
    CardID,
    Transaction_DateTime = DateTime,
    Amount, Store) |>
  arrange(Transaction_DateTime)

aubrey_statement
```

```
## # A tibble: 2 x 4
##   CardID           Transaction_DateTime Amount Store
##   <chr>            <dttm>                <dbl> <chr>
## 1 5628927579821287 2019-10-05 18:58:57   68.5  REI
## 2 4629183750293847 2019-10-16 14:30:21    4.98 Kickstand Kafe
```